

LAGRANGE

An autonomous Operations Research agent that turns a plain-English business question and a raw CSV into an optimal, explained decision.

Built on the **Cursor SDK** (Python, local runtime) with a deterministic OR tool belt exposed as custom tools. This document is the complete build plan: architecture, the data contract, the exact SDK surface to use, a three-way workstream split, an hour-by-hour schedule with integration gates, the demo script, and the fallback plan for when the venue Wi-Fi dies.

THE ONE THING THAT WINS THIS

Every team will demo a happy path: question in, numbers out. **We win on the unhappy path.** When a judge adds a constraint that makes the problem impossible, most agents print **INFEASIBLE** and die. Lagrange diagnoses *which business rules conflict*, computes the cheapest relaxation, and says: "Your 30 mpg sustainability target and your \$2M budget cannot both hold. Drop the target to 26.4 mpg, or raise the budget to \$2.31M." That is the moment people remember when they score.

RUNTIME

Cursor SDK · Python ·
local

MODEL

composer-2.5, pinned

SOLVER

OR-Tools CP-SAT / GLOP

TEAM

3 · Runtime, OR Core,
Product

01 Strategy: how the scoring actually works

The brief names four judging criteria. They are not equally easy to win. Translation fidelity and business explanation are where every team will land somewhere decent; **data ingestion autonomy** and **resilience & self-correction** are where teams either shine or quietly fail. We spend our engineering budget accordingly.

CRITERION	WHAT WE BUILD FOR IT	WHAT THE JUDGES SEE	OWNER	EFFORT
Translation fidelity	A typed <code>OptimizationSpec</code> where every constraint carries the exact sentence fragment it came from, plus an explicit <code>unmapped_phrases</code> list for anything the agent could not map.	A side-by-side traceability table: business sentence → mathematical equation. Nothing is silently dropped.	B C	High
Data ingestion autonomy	A <code>profile_dataset</code> tool that infers types, units, null strategy, cardinality, and nominates cost / value / category columns with confidence scores. No column names hardcoded anywhere.	We drop a CSV the agent has never seen (our wildcard) and it works, live.	B	High
Resilience & self-correction	A repair state machine with four distinct failure branches, backed by elastic-relaxation infeasibility diagnosis rather than blind retries.	Judge breaks the model on purpose. Agent explains <i>why</i> in business terms and proposes the minimal fix.	A B	Highest
Business explanation	Shadow prices and binding constraints turned into "what is this rule costing you" sentences, plus a one-screen executive brief.	An exec summary a CFO could read, with a "cost of each constraint" table nobody else will have.	C	Medium

Three rules we do not break

1. **Build the agent, not the answer.** The brief says explicitly it is not about solving one problem. Zero column names, zero dataset names, zero problem-specific branches in our code. If we ever type `"horsepower"` into a source file, we have lost a criterion.
2. **Generality is demonstrated, not claimed.** We prepare all three tracks *plus* a fourth wildcard dataset that we only reveal on stage. Claiming "it generalises" is worth nothing; showing it on unseen data is worth everything.
3. **Feature freeze at T–10h.** Hackathons are lost in the last four hours by teams still writing features. After the freeze we only fix, rehearse, and record.

SCOPE DISCIPLINE

Explicitly **out of scope**: user accounts, persistence beyond a run folder, multi-user, deployment, model comparison benchmarks, fancy charts beyond two, and any solver we have not already used. Every one of these has killed a hackathon team before.

02 Architecture

Five stages. Each stage is one `agent.send()` on a single durable Cursor SDK agent, so conversation context carries forward, but Python owns the sequencing. We deliberately do *not* hand orchestration to the model: a scripted pipeline is observable, instrumentable, and reproducible on stage.

STAGE 1 Profile Inspect the raw CSV. Infer dtypes, units, nulls, cardinality. Nominate candidate cost / value / capacity / category columns. → <code>schema_card.json</code>	STAGE 2 Formulate Map the business question onto the schema card. Choose LP / MIP / assignment. Emit typed spec with rule-to-equation provenance. → <code>spec.json</code>	STAGE 3 Compile & Solve Deterministic spec → OR-Tools model. Solve under a hard time limit. Return status, objective, variable values. → <code>solution.json</code>	STAGE 3B Diagnose & Repair On error / infeasible / unbounded / timeout: identify the conflicting rules, propose the minimal relaxation, re-solve. Max 3 attempts. → <code>diagnosis.json</code>	STAGE 4 Explain Solution + shadow prices → executive brief. Binding constraints become "what this rule costs you" sentences. → <code>brief.md</code>
---	--	---	---	--

The central design decision: a typed spec in the middle

The naive build is "agent writes Python, agent runs Python." It demos badly: the model regenerates slightly different code every run, so nothing is reproducible and the failure modes are unbounded. Instead the agent's job is to produce a **typed intermediate representation** — the `OptimizationSpec` — and a deterministic Python compiler we own turns that into an OR-Tools model.

WHAT THIS BUYS US

- Reproducible runs, so rehearsal is meaningful.
- The spec is a *renderable artifact* — it becomes the traceability table in the UI.
- Infeasibility diagnosis needs a structured model. You cannot elastically relax free-form code.
- Failures become typed and repairable instead of arbitrary tracebacks.

WHAT WE KEEP FROM FREE-FORM CODE

- The `shell` tool stays enabled. The agent can still write throwaway pandas to inspect the data.
- An `escape_hatch` spec field allows a raw expression when the schema genuinely cannot express something.
- This matters for the resilience criterion: we are structured by default, not restricted.

Repository layout

```
lagrange/  
├── orchestrator/  
│   ├── pipeline.py  
│   ├── prompts/  
│   ├── events.py  
│   └── replay.py  
├── orcore/  
│   ├── schema.py  
│   ├── profile.py  
│   ├── compile.py  
│   ├── solve.py  
│   ├── diagnose.py  
│   ├── sensitivity.py  
│   └── tools.py  
├── app/  
├── fixtures/  
├── data/  
└── tests/golden/  
# OWNER A – Cursor SDK runtime  
# stage sequencing, retries, error taxonomy  
# one .md per stage, version-controlled  
# normalised event bus → UI (SSE)  
# deterministic playback of a recorded run  
# OWNER B – deterministic OR toolbelt  
# OptimizationSpec dataclasses + JSON Schema  
# CSV → schema card  
# spec → OR-Tools model  
# solve + status normalisation  
# elastic relaxation / conflicting rule set  
# duals, binding constraints, reduced costs  
# CustomTool wrappers (the agent-facing surface)  
# OWNER C – UI + report rendering  
# frozen schema cards, specs, recorded runs  
# local CSV copies – NEVER fetch live in the demo  
# spec → expected objective, per track
```

03 The contract: OptimizationSpec

This is the most important artifact in the project. It is what the agent produces, what the compiler consumes, what the UI renders, and what wins the translation-fidelity criterion.

INTERFACE FREEZE

This schema and the event JSON in `orchestrator/events.py` are frozen at Gate 1 (H+3). After that, changes require all three people to agree, because everyone is coding against fixtures derived from them. This single rule is what lets three people work in parallel without blocking each other.

```
{
  "problem_class": "mixed_integer_linear",          // lp | mixed_integer_linear | assignment | scheduling
  "sense": "maximize",
  "index_sets": {
    "I": { "source": "rows", "filter": "cylinders <= 8", "limit": 400 }
  },
  "parameters": {
    "value": { "column": "horsepower", "na_strategy": "drop_row", "unit": "hp" },
    "cost": { "column": "weight", "transform": "x * 9.4", "unit": "usd",
              "derivation": "no price column present; proxied from weight" },
    "effic": { "column": "mpg", "na_strategy": "median", "unit": "mpg" }
  },
  "variables": [
    { "name": "x", "index": "I", "domain": "integer", "lb": 0, "ub": 12,
      "meaning": "units of vehicle model i to purchase" }
  ],
  "objective": {
    "expr": "sum(value[i] * x[i] for i in I)",
    "business_meaning": "total fleet engine power"
  },
  "constraints": [
    { "id": "budget",
      "expr": "sum(cost[i] * x[i] for i in I) <= 2000000",
      "source_phrase": "a fixed capital budget of two million",    // <-- provenance
      "elastic": false, "priority": "hard" },
    { "id": "fleet_mpg",
      "expr": "sum((effic[i] - 28) * x[i] for i in I) >= 0",
      "source_phrase": "strict corporate sustainability targets",
      "elastic": true, "relax_cost": 1.0, "priority": "soft" }
  ],
  "assumptions": [
    "'sustainability target' interpreted as fleet-average mpg >= 28 (industry default)"
  ],
  "unmapped_phrases": [
    "'prefer European manufacturers' - no manufacturer-region column in this dataset"
  ],
  "clarifying_questions": [
    "Should the mpg target apply to the fleet average or to every vehicle individually?"
  ]
}
```

Why the four unusual fields matter

FIELD	WHY IT EXISTS
source_phrase	Every equation traces back to the words the user wrote. This renders directly as the traceability table — the single clearest evidence of translation fidelity a judge can be shown.
elastic + relax_cost	Marks which rules are negotiable. Without this, infeasibility diagnosis has no idea which constraint it is allowed to bend, and the whole differentiator collapses.

FIELD	WHY IT EXISTS
unmapped_phrases	Honest reporting of what could not be modelled. Judges trust an agent that admits limits far more than one that silently ignores half the prompt.
derivation	When no cost column exists, the agent proxies one and says so. This is exactly the "messy reality" the brief describes.

SMALL FEATURE, LARGE PAYOFF

Because `source_phrase` exists, Owner C can render a two-column table — *what you said* beside *what we solved* — in about twenty minutes of work. It will be the screenshot that ends up in the judges' notes.

04 The Cursor SDK surface we use

Setup

```
uv pip install cursor-sdk ortools pandas pyarrow fastapi uvicorn pydantic
export CURSOR_API_KEY="crsr_..." # Cursor Dashboard -> Integrations
```

Python, not TypeScript. OR-Tools, pandas and every solver binding live in Python. Putting the orchestrator in the same language as the tool belt means custom tools are ordinary function references rather than a subprocess boundary. Requires Python 3.10+.

Local runtime, not cloud. The datasets sit on disk, the demo must run without depending on venue Wi-Fi for file access, and custom tools are a local-only feature. Pass `local=` explicitly — the SDK defaults to local when neither `local` nor `cloud` is set, and silently getting the wrong runtime is the single most common SDK mistake.

Which SDK features, and why

SDK FEATURE	HOW WE USE IT	WHY IT EARNS ITS PLACE
<code>Agent.create()</code> + <code>agent.send()</code>	One durable agent per business question; each pipeline stage is a separate <code>send()</code> .	Stage 4 can reference what stage 2 decided without us re-serialising context by hand.
<code>local.custom_tools</code>	The entire OR tool belt: profile, validate, compile-and-solve, diagnose, sensitivity, scenario.	Plain Python functions become agent tools. No MCP server to stand up, no subprocess, no wire format. This is the highest-leverage feature in the whole SDK for us.
<code>tools=[...]</code> / <code>disallowed_tools=[...]</code>	Per-stage toolset narrowing: the Formulator gets no <code>shell</code> ; the Explainer gets read-only.	A stage that cannot reach for the wrong tool cannot wander. Cuts latency and failure surface noticeably.
<code>mode="plan"</code> → <code>SendOptions(mode="agent")</code>	Formulation runs in plan mode and surfaces clarifying questions before anything executes.	Demonstrates a human-in-the-loop checkpoint for a non-technical stakeholder — directly on-brief.
<code>SendOptions(on_delta=..., on_step=...)</code>	Push token deltas and tool-call steps to the UI event bus as they happen.	The live "agent thinking" view. Cheap to wire, disproportionately convincing on stage.
<code>run.wait()</code> → <code>RunResult</code>	Called after every stage without exception.	The only way to know a stage finished versus errored versus was cancelled. Skipping it also leaks watchers.
<code>CursorAgentError</code> vs <code>result.status == "error"</code>	Two separate handlers: never-started (auth/network, retry) versus started-and-failed (inspect and repair).	Conflating them means the repair loop retries auth failures forever while the venue Wi-Fi flaps.

SDK FEATURE	HOW WE USE IT	WHY IT EARNS ITS PLACE
<code>Agent.resume(agent_id)</code>	Judge Q&A after the pipeline finishes: "what if the budget were 20% higher?"	Turns the demo from a recording into a conversation. Re-pass custom tools — they are not persisted.
<code>agents={...}</code> (<code>AgentDefinition</code>)	One optional subagent, <code>alternative-formulator</code> , to explore a second model shape in parallel.	<i>Stretch goal only.</i> Do not build this before Gate 3.
<code>agent.get_usage()</code>	Report tokens and dollars per decision in the closing slide.	"This decision cost 34 cents and 90 seconds" lands hard with a finance audience.
<code>Cursor.models.list()</code>	Run once at setup to confirm <code>composer-2.5</code> is available on the team key.	Thirty seconds now, versus discovering a bad model ID at hour eighteen.

DO NOT USE

Cloud runtime (custom tools are local-only; needs a cloned repo). **Artifacts API** (`list_artifacts()` returns empty on local agents — read the run folder instead). **Ambient settings** — leave `setting_sources` unset so nobody's personal `~/.cursor` config leaks into a run and makes it irreproducible on a different laptop.

05 The tool belt

The agent sees two kinds of tools: Cursor's built-ins, narrowed per stage, and our six custom OR tools. The custom tools are where all determinism lives.

Custom tools (LocalAgentOptions.custom_tools) B owns

TOOL	SIGNATURE	RETURNS / PURPOSE
profile_dataset	(path, sample_rows=2000)	Schema card: per-column dtype, null rate, min/max/quantiles, cardinality, inferred unit, and a ranked nomination of which columns look like cost, value, capacity, and category — each with a confidence score. This tool is the data-ingestion-autonomy criterion.
validate_spec	(spec)	JSON-Schema check plus semantic lint: undefined parameters, index sets referenced but not declared, an objective with no variables, unit mismatch across a comparison, a spec with zero constraints. Returns actionable messages the agent can act on, not a stack trace.
compile_and_solve	(spec, time_limit_s=10)	Deterministic spec → OR-Tools model → solve. Normalised status (OPTIMAL , FEASIBLE , INFEASIBLE , UNBOUNDED , TIMEOUT , COMPILE_ERROR), objective value, non-zero variable assignments, MIP gap, wall time. Always returns the best incumbent on timeout.
diagnose_infeasibility	(spec)	Adds a non-negative slack to every elastic constraint, minimises the weighted sum of slacks, and reports the minimal set of rules that must bend and by exactly how much. This tool is the resilience criterion.
sensitivity_report	(spec, solution)	Shadow prices and binding constraints from the LP relaxation, plus reduced costs for near-miss items. Feeds the "what each rule costs you" section of the brief.
run_scenario	(spec, overrides)	Re-solve with patched numbers. Powers live judge Q&A and the tornado chart.

Built-in Cursor tools, per stage A owns

STAGE	BUILT-INS OFFERED	RATIONALE
1 · Profile	read , ls , glob , shell	Shell stays on so the agent can run ad-hoc pandas when the profiler's nominations look wrong. Genuine ingestion autonomy.

STAGE	BUILT-INS OFFERED	RATIONALE
2 · Formulate	<code>read</code> , <code>grep</code> · <code>disallowed_tools=["shell","edit"]</code>	Formulation is a reasoning task. Denying execution stops the agent from "just writing a script" and bypassing the spec.
3 · Solve / repair	<code>read</code> , <code>write</code> , <code>edit</code> , <code>shell</code>	Full latitude for the escape hatch and for inspecting solver output.
4 · Explain	<code>read</code> , <code>write</code>	Writing prose. Nothing else needed.

CUSTOM TOOLS OR AN MCP SERVER?

Use `custom_tools` for the hackathon: a Python function reference, zero transport, and edits take effect on the next run. If you also want the tool belt usable from the Cursor IDE while developing (genuinely handy for Owner B), wrap the same functions in a small stdio MCP server later and pass it via `mcp_servers={"orcure": StdioMcpServerConfig(command="python", args=["-m","orcure.mcp"] )}`. Write the functions once, expose them twice. Do not do this before Gate 3.

06 The self-correction loop

Blind retry is what everyone else will build, and it fails visibly on stage. We branch on the *kind* of failure, because each kind has a different correct response.

STATUS	DIAGNOSIS STEP	REPAIR ACTION	MAX
COMPILE_ERROR	Return the <code>validate_spec</code> message plus the offending JSON path.	Agent patches only that field and revalidates. No full regeneration.	3
INFEASIBLE	<code>diagnose_infeasibility</code> → minimal conflicting rule set with required slack per rule.	Two options presented: relax the named soft rule by the computed amount, or report the conflict to the user with the numbers. Never silently relax a rule marked hard.	2
UNBOUNDED	Find variables with no upper bound appearing in the objective with a favourable sign.	Infer a bound from the data (e.g. available stock, budget ÷ unit cost) and record it as an explicit assumption.	2
TIMEOUT	Report incumbent objective and MIP gap.	Tighten the index set via <code>limit</code> , relax integrality where it is not semantically required, or accept the incumbent and say so.	1

Elastic relaxation, concretely

For each `elastic` constraint, introduce slack $s_c \geq 0$, rewrite $g_c(x) \leq b_c$ as $g_c(x) \leq b_c + s_c$, and minimise $\sum w_c \cdot s_c$. Every $s_c > 0$ in the solution names a rule that must bend and by how much. Hard constraints get no slack, so they are never quietly violated.

THE DEMO MOMENT — REHEARSE THIS EXACT WORDING

"Your sustainability floor of 30 mpg conflicts with your \$2M budget. They cannot both hold. The smallest change that makes this solvable is lowering the fleet target to 26.4 mpg — or, if the target is fixed, raising the budget to \$2.31M. Every other rule you gave me is satisfied. Which would you like?"

Nobody demos this. Everybody demos a bar chart.

Failure taxonomy at the SDK layer A owns

Log `run.id` and `agent.agent_id` *before* you start streaming. If a stream hangs at hour twenty, those two IDs are the only way to find out what happened.

```
try:
    run = agent.send(prompt, opts)
    log.info("run", run_id=run.id, agent_id=agent.agent_id) # log BEFORE streaming
    result = run.wait()
    if result.status == "error":
        return StageOutcome.retryable(result) # executed, then failed -> repair
except CursorAgentError as err:
    if err.is_retryable:
        sleep(err.retry_after or backoff())
        return StageOutcome.retry_transport()
    raise
```

07 Orchestrator skeleton

Enough to be a real starting point. Owner A should have this running against stub tools by H+5.

```
# orchestrator/pipeline.py
import os, json, pathlib
from cursor_sdk import (
    Agent, AgentOptions, LocalAgentOptions, SendOptions,
    CustomTool, CursorAgentError,
)
from orcore import tools as T

RUN_DIR = pathlib.Path("runs") / run_id
MODEL = "composer-2.5" # pinned. never "auto" during the demo.

def tool(fn, description, schema):
    return CustomTool(description=description, input_schema=schema, execute=fn)

OR_TOOLS = {
    "profile_dataset": tool(T.profile_dataset, T.D_PROFILE, T.S_PROFILE),
    "validate_spec": tool(T.validate_spec, T.D_VALIDATE, T.S_SPEC),
    "compile_and_solve": tool(T.compile_and_solve, T.D_SOLVE, T.S_SOLVE),
    "diagnose_infeasibility": tool(T.diagnose_infeasibility, T.D_DIAGNOSE, T.S_SPEC),
    "sensitivity_report": tool(T.sensitivity_report, T.D_SENS, T.S_SENS),
    "run_scenario": tool(T.run_scenario, T.D_SCENARIO, T.S_SCEN),
}

def run_pipeline(question: str, csv_path: str, bus):
    opts = AgentOptions(
        model=MODEL,
        api_key=os.environ["CURSOR_API_KEY"], # explicit, not ambient
        name=f"lagrange-{run_id}",
        mode="plan", # formulate before acting
        disallowed_tools=["webSearch"], # nothing to look up; saves latency
        local=LocalAgentOptions(
            cwd=str(RUN_DIR), # sandboxed per run
            custom_tools=OR_TOOLS,
            # setting_sources deliberately unset: inline config only
        ),
    )

    with Agent.create(opts) as agent:
        bus.emit("agent_created", agent_id=agent.agent_id)

        card = stage(agent, bus, "profile", P.PROFILE.format(csv=csv_path))
        spec = stage(agent, bus, "formulate", P.FORMULATE.format(q=question, card=card))

        # plan -> agent mode once the human (or auto-accept) confirms the spec
        sol = stage(agent, bus, "solve", P.SOLVE, mode="agent")

        attempts = 0
        while sol["status"] in REPAIRABLE and attempts < MAX_REPAIRS[sol["status"]]:
            attempts += 1
            bus.emit("repair_attempt", n=attempts, status=sol["status"])
            sol = stage(agent, bus, "repair", P.REPAIR[sol["status"]].format(**sol), mode="agent")

        brief = stage(agent, bus, "explain", P.EXPLAIN, mode="agent")

        usage = agent.get_usage()
        bus.emit("done", tokens=usage.usage.total_tokens,
            cents=usage.cost.charged_cents if usage.cost else None)
        return brief

def stage(agent, bus, name, prompt, mode=None):
    opts = SendOptions(
        mode=mode,
        on_delta=lambda u: bus.delta(name, u),
        on_step=lambda s: bus.step(name, s),
    )
```

```

)
run = agent.send(prompt, opts)
bus.emit("stage_start", stage=name, run_id=run.id)
result = run.wait() # ALWAYS
if result.status != "finished":
    raise StageFailed(name, result.status, run.id)
return read_artifact(RUN_DIR, name) # stage writes JSON to the run dir

```

Custom tool definition B owns

```

# orcore/tools.py
D_DIAGNOSE = (
    "Explain WHY an optimization model is infeasible. Adds slack to every constraint "
    "marked elastic, minimizes total weighted slack, and returns the minimal set of "
    "business rules that must be relaxed and by exactly how much. Call this immediately "
    "whenever compile_and_solve returns INFEASIBLE. Never guess at a relaxation yourself."
)

def diagnose_infeasibility(args, context):
    spec = OptimizationSpec.model_validate(args["spec"])
    report = diagnose.elastic_relax(spec)
    return {
        "feasible_after_relaxation": report.solved,
        "violated_rules": [
            {"id": v.id, "source_phrase": v.source_phrase,
             "required_slack": v.slack, "human": v.sentence()}
            for v in report.violations
        ],
        "hard_conflicts": report.hard_conflicts,
    }

```

TOOL DESCRIPTIONS ARE PROMPT ENGINEERING

The description string is the only thing the model reads when deciding whether to call a tool. Spend real effort there. Note how `D_DIAGNOSE` states the trigger condition ("whenever `compile_and_solve` returns `INFEASIBLE`") and the anti-behaviour ("never guess at a relaxation yourself"). That one sentence is worth more than a page of system prompt.

08 Three people, three workstreams

Split by *architectural layer*, not by feature, so the seams are few and typed. Each person owns a directory nobody else edits; cross-layer changes go through the frozen contract.

A Agent Runtime — orchestrator/

The Cursor SDK specialist. Owns everything between the user's question and the tool belt.

Deliverables. Staged pipeline with per-stage toolset narrowing · version-controlled prompt files · the four-branch repair state machine · error taxonomy separating `CursorAgentError` from `result.status == "error"` · normalised event bus feeding the UI · `replay.py` for network-free playback · usage and cost telemetry · `Agent.resume` path for live Q&A.

Depends on. Tool signatures from B (stubs are fine from H+3) · event schema agreed with C.

Done when. A single command takes a question plus a CSV path and produces `brief.md`, with the full event stream on stdout, and survives an injected INFEASIBLE.

B OR Core — orcore/

The optimisation engineer. Owns everything the agent calls. Zero Cursor SDK knowledge required.

Deliverables. `OptimizationSpec` dataclasses and JSON Schema · CSV profiler with column-role nomination · spec→OR-Tools compiler · solve with status normalisation and hard time limits · elastic-relaxation diagnosis · sensitivity and shadow prices · six `CustomTool` wrappers with carefully written descriptions · golden tests with known optima.

Depends on. Nothing. B can start at minute zero and should be furthest ahead by Gate 1.

Done when. `pytest tests/golden` is green for all three tracks and `diagnose_infeasibility` returns the correct violated rule for a deliberately over-constrained spec.

C Product & Story — app/, deck, demo

The one who decides what the judges actually see. This role is undervalued and decides the winner.

Deliverables. Single-page UI: question box, CSV drop, live agent stream, traceability table, solution table, executive brief · the rule-to-equation traceability view (highest-value screen we have) · brief renderer with the "cost of each constraint" section · all four datasets downloaded locally · **the wildcard dataset** · the deck · the demo script and its timing · the recorded fallback video.

Depends on. Event schema and spec fixtures only. Builds against `fixtures/` from H+1; never waits for a working pipeline.

Done when. The demo runs end to end in under six minutes, three times consecutively, on the presenting laptop.

Working agreements

- Commit hourly, straight to `main`. No PRs, no branches. Broken `main` stops the line and everyone fixes it.

- **Fixtures are the contract.** B commits a real `schema_card.json` and `spec.json` for Track A by H+2; A and C build against those files, not against each other's code.
- **Standup on the hour, ninety seconds.** Shipped, next, blocked.
- **Blocked fifteen minutes is a team problem.** Say it out loud. Nobody debugs alone for an hour in a hackathon.
- **C has veto over scope.** If it will not be seen in the six-minute demo, it does not get built.

09 Hour-by-hour (24-hour schedule)

Hours are relative to kickoff. For a 12-hour event, halve everything after H+3 and drop Track C. For a 36-hour event, do not add features — add rehearsals and sleep.

WINDOW	A RUNTIME	B OR CORE	C PRODUCT
H+0–1	<i>All three together.</i> Read the brief aloud. Agree the <code>OptimizationSpec</code> fields on a whiteboard. Agree the event JSON. Create the repo, install deps, verify <code>CURSOR_API_KEY</code> works with a five-line <code>Agent.prompt</code> <code>hello-world</code> , confirm <code>composer-2.5</code> via <code>Cursor.models.list()</code> . Do not skip the key check.		
H+1–3	Agent creation, one stage end to end with a fake tool. Event bus emitting.	<code>schema.py</code> + profiler. Commit Track A schema card and a hand-written spec to <code>fixtures/</code> .	Download all four CSVs to <code>data/</code> . UI shell rendering the fixture spec. Deck outline.
GATE 1 · H+3 · Contract frozen. Spec schema and event JSON committed. Everyone unblocked. No further schema changes without a three-way agreement.			
H+3–8	All five stages wired with stub tools. Per-stage toolsets. Prompt files v1.	Compiler + solve. Track A produces a correct objective. Golden test #1 green.	Live stream view. Traceability table. Solution table.
GATE 2 · H+8 · First real end-to-end run on Track A. Ugly output is fine. If this slips past H+10, cut Track C immediately and say so out loud.			
H+8–14	Repair state machine, all four branches. Injected-failure tests. Structured logging.	<code>diagnose_infeasibility</code> + <code>sensitivity_report</code> . Track B green.	Executive brief renderer. Constraint-cost section. Deck v1 complete.
GATE 3 · H+14 · Feature freeze. Nothing new after this point. Remaining time is Track C, the wildcard, hardening, and rehearsal. Subagents and the MCP server are cut unless everything else is genuinely done.			
H+14–18	<code>replay.py</code> . <code>Agent.resume</code> Q&A path. Time and stabilise the full run.	Track C. Wildcard dataset dry run. Fix whatever the wildcard breaks.	Record the fallback video. Polish. Full rehearsal #1.
GATE 4 · H+18 · Code freeze. Demo-blocking bugs only. Deck final. Everything from here is rehearsal.			
H+18–21	<i>All three.</i> Rehearsals #2, #3, #4 — on the presenting laptop, on the venue network if possible, with someone playing a hostile judge. Time every run. Fix only what breaks the demo.		
H+21–24	Buffer and sleep. Final rehearsal thirty minutes before the slot. Laptop charged, adapter tested, video on the desktop, replay mode verified working with Wi-Fi turned off.		

THE CLASSIC FAILURE

Three people build three excellent components and integrate at H+20. It does not work, and there is no time to find out why. Gate 2 exists specifically to prevent this: **one ugly end-to-end run by H+8 is worth more than three beautiful components at H+16.**

10 The six-minute demo

Rehearse until the wording is muscle memory. The person presenting is not the person driving the laptop.

- 0:00** **The problem, one sentence.** "A bank has the data and the question. Between them sits an OR specialist and three weeks. We removed both." No slide about what operations research is — the judges wrote the brief.
- 0:30** **Paste a deliberately messy question.** Vague, contradictory, includes one requirement the data cannot support. Drop the CSV. Hit run. Say nothing while it starts — let the live stream carry the moment.
- 1:00** **Schema card.** "It has never seen this file. It inferred that this column is the cost, this one is capacity, and that 6% of rows have a null it will drop." Point at the confidence scores.
- 1:40** **Traceability table.** The highest-value screen we have. Your words on the left, the equations on the right. Then: "And here is what it could *not* map — it is telling us the data has no manufacturer region, so it asked instead of guessing."
- 2:40** **Solved.** Executive brief on screen. Read one sentence of it aloud. Then the constraint-cost table: "every extra mpg of the sustainability target costs 340 horsepower of fleet capacity." Nobody else will have that number.
- 3:30** **THE MOMENT.** Turn to the judges: "*Give me a rule that breaks it.*" Type their rule in. It goes INFEASIBLE. Silence for two seconds. Then the diagnosis appears: which rules conflict, by how much, and the two cheapest ways out. This is the whole pitch.
- 5:00** **The wildcard.** A dataset and a domain we have never mentioned, from a different track entirely. Same agent, no code changes. Run it. "We did not build a fleet optimiser. We built the thing that builds fleet optimisers."
- 5:30** **Close on numbers.** One architecture slide, then: 90 seconds and 34 cents per decision, versus three weeks of specialist time. Stop talking.

Fielding the three questions you will be asked

QUESTION	ANSWER
"How do you know the model it wrote is correct?"	Three layers: the spec is schema-validated and semantically linted before anything runs; every constraint traces to a source phrase a human can check in seconds; and the sensitivity report exposes which constraints actually bind, which is where a wrong model shows itself immediately. Plus golden tests with known optima on all three tracks.
"What happens on a problem class you did not anticipate?"	The spec covers LP, MIP, assignment and scheduling shapes. Outside that, the escape hatch lets the agent emit a raw expression and the shell tool lets it write bespoke code — it degrades to a general coding agent rather than failing. Show the wildcard again if there is time.
"Is this just a wrapper around an LLM?"	The LLM does translation. The mathematics is deterministic OR-Tools code we wrote, so the optimum is provably optimal, not generated. The interesting engineering is the repair loop — turning solver failure states into business language — and that is not a prompt, it is elastic relaxation.

11 Risks and fallbacks

RISK	IMPACT	MITIGATION — DO THIS BEFORE IT HAPPENS
Venue network dies mid-demo	Fatal	<code>replay.py</code> plays a recorded event stream with identical UI and zero network. Rehearse the demo once with Wi-Fi physically off. Also have the recorded video on the desktop as a second fallback.
All CSVs are fetched from GitHub at runtime	Fatal	Download all four to <code>data/</code> in hour one. Never fetch a URL during the demo. Ever.
Model output varies between runs	High	Pin <code>composer-2.5</code> — never <code>auto</code> . Keep prompts in version-controlled files, not string literals. Keep golden fixtures so you can always fall back to a known-good spec.
Solver hangs on a large MIP	High	Hard 10-second limit inside <code>compile_and_solve</code> , always return the incumbent, and cap index sets via <code>limit</code> . Never let the solver own the clock during a live demo.
API key rate limits or expires	High	Verify the key in hour one. Have a second key from a second account ready in the environment. Know the cost per run from <code>get_usage()</code> so the budget holds no surprises.
Integration slips past H+10	High	Gate 2 is a hard checkpoint. If it slips, cut Track C and the sensitivity report immediately, on the spot, out loud.
The wildcard dataset breaks the agent	Medium	That is exactly why we test it at H+14, not on stage. If it still breaks at H+18, cut the wildcard from the demo and lean harder on the infeasibility moment.
Someone builds subagents / MCP server / a second UI	Medium	Gate 3 feature freeze. C has veto. All three are explicitly listed as stretch goals in this document precisely so they can be cut without argument.

Cut list, in the order things get cut

- 1. Subagents (`agents={...}`) and the standalone MCP server — cut first, no discussion.
- 2. Track C (scheduling) — the assignment formulation is the most work for the least visible payoff.
- 3. The tornado / what-if chart — the number in a sentence is nearly as good.
- 4. Sensitivity report — keep binding constraints, drop reduced costs.
- 5. The wildcard dataset — painful, but the demo survives without it.

Never cut: the traceability table, and the infeasibility diagnosis. Those two *are* the project.

IF YOU REMEMBER ONE THING

Ship an ugly end-to-end pipeline by hour eight, then spend the remaining sixteen hours making one failure mode beautiful. Teams that lose spend twenty-two hours making four things good and never get to show any of them working together.